

Zadanie 1

Otwarto: poniedziałek, 17 marca 2025, 00:00 **Wymagane do:** niedziela, 13 kwietnia 2025, 23:59

Źródło: serwis MIMoodle Wydziału Matematyki, Informatyki i Mechaniki Uniwersytetu Warszawskiego, kurs AKSO.24/25L — moodle.mimuw.edu.pl/mod/assign/view.php?id=149769. Autorami treści zadania i załączonych plików są prowadzący przedmiot.

Automaty Moore'a

Zadanie polega na zaimplementowaniu w języku C dynamicznie ładowanej biblioteki symulującej automaty Moore'a. Automat Moore'a jest to rodzaj deterministycznego automatu skończonego stosowanego w synchronicznych układach cyfrowych. Automat Moore'a reprezentujemy jako uporządkowaną szóstkę $\langle X, Y, Q, t, y, q \rangle$, gdzie

- X jest zbiorem wartości przyjmowanych przez sygnały wejściowe,
- Y jest zbiorem wartości przyjmowanych przez sygnały wyjściowe,
- Q jest zbiorem wartości stanów wewnętrznych,
- $t: X \times Q \rightarrow Q$ jest funkcją przejść,
- $y: Q \rightarrow Y$ jest funkcją wyjść,
- $q \in Q$ jest stanem początkowym.

Rozważamy tylko automaty binarne, które mają n jednobitowych sygnałów wejściowych, m jednobitowych sygnałów wyjściowych, a stan ma s bitów. Formalnie $X = \{0, 1\}^n$, $Y = \{0, 1\}^m$, $Q = \{0, 1\}^s$.

W każdym kroku działania automatu funkcja t na podstawie wartości sygnałów wejściowych i aktualnego stanu automatu wylicza nowy stan automatu. **Funkcja y na podstawie stanu automatu wylicza wartości sygnałów wyjściowych.**

Interfejs biblioteki

Interfejs biblioteki znajduje się w załączonym do treści zadania pliku `ma.h`. Dodatkowe szczegóły działania biblioteki należy wywnioskować z załączonego do treści zadania przykładu użycia `ma_example.c`, który jest integralną częścią specyfikacji.

Ciąg bitów i wartości sygnałów przechowujemy w tablicy, której elementy są typu `uint64_t`. W jednym elemencie przechowujemy kolejne 64 bity, począwszy od najmniej znaczącej pozycji. Jeśli długość ciągu nie jest wielokrotnością 64, to bardziej znaczące bity ostatniego elementu tablicy pozostają nieużywane.

Typ strukturalny `moore_t` reprezentuje automat. Typ ten trzeba zdefiniować (zaimplementować) w ramach tego zadania.

```
typedef struct moore moore_t;
```

Stan automatu oraz sygnały na jego wejściach i wyjściach są ciągami bitów.

Typ `transition_function_t` reprezentuje funkcję przejść automatu. Funkcja ta oblicza nowy stan automatu na podstawie sygnałów wejściowych i aktualnego stanu automatu.

```
typedef void (*transition_function_t)(uint64_t *next_state, uint64_t const *input,
    uint64_t const *state, size_t n, size_t s);
```

Parametry funkcji:

- `next_state` – wskaźnik na ciąg bitów, w którym jest umieszczany nowy stan automatu;
- `input` – wskaźnik na ciąg bitów zawierający wartości sygnałów wejściowych;
- `state` – wskaźnik na ciąg bitów zawierający aktualny stan automatu;
- `n` – liczba sygnałów wejściowych automatu;
- `s` – liczba bitów stanu wewnętrznego automatu.

Typ `output_function_t` reprezentuje funkcję wyjść automatu. Funkcja ta oblicza sygnały na wyjściach automatu na podstawie stanu automatu.

```
typedef void (*output_function_t)(uint64_t *output, uint64_t const *state,
    size_t m, size_t s);
```

Parametry funkcji:

- `output` – wskaźnik na ciąg bitów, w którym jest umieszczany wynik funkcji;
- `state` – wskaźnik na ciąg bitów zawierający stan automatu;
- `m` – liczba sygnałów wyjściowych automatu;
- `s` – liczba bitów stanu wewnętrznego automatu.

Funkcja `ma_create_full` tworzy nowy automat.

```
moore_t * ma_create_full(size_t n, size_t m, size_t s, transition_function_t t,
    output_function_t y, uint64_t const *q);
```

Parametry funkcji:

- `n` – liczba sygnałów wejściowych automatu;
- `m` – liczba sygnałów wyjściowych automatu;
- `s` – liczba bitów stanu wewnętrznego automatu;
- `t` – funkcja przejść automatu;
- `y` – funkcja wyjść automatu;
- `q` – wskaźnik na ciąg bitów reprezentujący początkowy stan automatu.

Wynik funkcji:

- wskaźnik na strukturę reprezentującą automat;
- `NULL` – jeśli któryś z parametrów `m` lub `s` jest równy zero albo któryś ze wskaźników `t`, `y`, `q` ma wartość `NULL`, albo wystąpił błąd alokowania pamięci; funkcja ustawia wtedy `errno` odpowiednio na `EINVAL` lub `ENOMEM`.

Funkcja `ma_create_simple` tworzy nowy automat, w którym liczba wyjść jest równa liczbie bitów stanu, funkcja wyjść jest identycznością, a stan początkowy jest wyzerowany. **Inicjuje zerami nieużywane bity stanu automatu.**

```
moore_t * ma_create_simple(size_t n, size_t s, transition_function_t t);
```

Parametry funkcji:

- `n` – liczba sygnałów wejściowych automatu;
- `s` – liczba bitów stanu wewnętrznego automatu i liczba sygnałów wyjściowych automatu;
- `t` – funkcja przejść automatu.

Wynik funkcji:

- wskaźnik na strukturę reprezentującą automat;
- `NULL` – jeśli parametr `s` jest równy zero albo wskaźnik `t` ma wartość `NULL`, albo wystąpił błąd alokowania pamięci; funkcja ustawia wtedy `errno` odpowiednio na `EINVAL` lub `ENOMEM`.

Funkcja `ma_delete` usuwa wskazany automat i zwalnia całą używaną przez niego pamięć. Nic nie robi, jeśli zostanie wywołana ze wskaźnikiem `NULL`. Po wykonaniu tej funkcji przekazany jej wskaźnik staje się nieważny.

```
void ma_delete(moore_t *a);
```

Parametr funkcji:

- `a` – wskaźnik na strukturę reprezentującą automat.

Funkcja `ma_connect` podłącza kolejne `num` sygnałów wejściowych automatu `a_in` do sygnałów wyjściowych automatu `a_out`, począwszy od sygnałów o numerach odpowiednio `in` i `out`, ewentualnie odłączając wejścia od innych wyjść, jeśli były podłączone.

```
int ma_connect(moore_t *a_in, size_t in, moore_t *a_out, size_t out, size_t num);
```

Parametry funkcji:

- `a_in` – wskaźnik na strukturę reprezentującą automat;
- `in` – numer wejścia automatu `a_in`;
- `a_out` – wskaźnik na strukturę reprezentującą automat;
- `out` – numer wyjścia automatu `a_out`;
- `num` – liczba łączonych sygnałów.

Wynik funkcji:

- `0` – jeśli operacja zakończyła się sukcesem;
- `-1` – jeśli któryś wskaźnik ma wartość `NULL`, parametr `num` jest równy zero lub wskazany zakres numerów wejść lub wyjść jest niepoprawny, albo wystąpił błąd alokowania pamięci; funkcja ustawia wtedy `errno` odpowiednio na `EINVAL` lub `ENOMEM`.

Funkcja `ma_disconnect` odłącza kolejne `num` sygnałów wejściowych automatu `a_in` od sygnałów wyjściowych, począwszy od wejścia o numerze `in`. Jeśli któreś wejście nie było podłączone, to pozostaje niepodłączone.

```
int ma_disconnect(moore_t *a_in, size_t in, size_t num);
```

Parametry funkcji:

- `a_in` – wskaźnik na strukturę reprezentującą automat;
- `in` – numer wejścia automatu `a_in`;
- `num` – liczba rozłączanych sygnałów.

Wynik funkcji:

- 0 – jeśli operacja zakończyła się sukcesem;
- -1 – jeśli wskaźnik ma wartość `NULL`, parametr `num` jest równy zero lub wskazany zakres numerów wejść jest niepoprawny; funkcja ustawia wtedy `errno` na `EINVAL`.

Funkcja `ma_set_input` ustawia wartości sygnałów na niepodłączonych wejściach automatu, ignorując w ciągu `input` bity odpowiadające wejściom podłączonym.

```
int ma_set_input(moore_t *a, uint64_t const *input);
```

Parametry funkcji:

- `a` – wskaźnik na strukturę reprezentującą automat;
- `input` – wskaźnik na ciąg `n` bitów, gdzie `n` jest liczbą wejść automatu.

Wynik funkcji:

- 0 – jeśli operacja zakończyła się sukcesem;
- -1 – jeśli automat nie ma wejść lub któryś wskaźnik ma wartość `NULL`; funkcja ustawia wtedy `errno` na `EINVAL`.

Funkcja `ma_set_state` ustawia stan automatu.

```
int ma_set_state(moore_t *a, uint64_t const *state);
```

Parametry funkcji:

- `a` – wskaźnik na strukturę reprezentującą automat;
- `state` – wskaźnik na ciąg bitów, nowy stan automatu do ustawienia.

Wynik funkcji:

- 0 – jeśli operacja zakończyła się sukcesem;
- -1 – jeśli któryś wskaźnik ma wartość `NULL`; funkcja ustawia wtedy `errno` na `EINVAL`.

Funkcja `ma_get_output` zwraca wskaźnik, pod którym przechowywany jest ciąg bitów zawierający wartości sygnałów na wyjściu automatu. Wskaźnik ten powinien pozostawać ważny do czasu wywołania funkcji `ma_delete` na tym automacie.

```
uint64_t const * ma_get_output(moore_t const *a);
```

Parametr funkcji:

- `a` – wskaźnik na strukturę reprezentującą automat.

Wynik funkcji:

- wskaźnik na ciąg bitów zawierający wartości sygnałów na wyjściu automatu lub `NULL`, gdy `a` ma wartość `NULL` – funkcja ustawia wtedy `errno` na `EINVAL`.

Funkcja `ma_step` wykonuje jeden krok obliczeń podanych automatów. Wszystkie automaty działają równolegle i synchronicznie, czyli wartości stanów i wyjść po wywołaniu funkcji `ma_step` zależą tylko od wartości stanów, wejść i wyjść przed wywołaniem tej funkcji.

```
int ma_step(moore_t *at[], size_t num);
```

Parametr funkcji:

- `at` – tablica wskaźników na strukturę reprezentującą automat;
- `num` – rozmiar tablicy `a`, liczba automatów.

Wynik funkcji:

- `0` – jeśli operacja zakończyła się sukcesem;
- `-1` – jeśli podany wskaźnik `a` ma wartość `NULL`, któryś ze wskaźników w tablicy `a` ma wartość `NULL` lub `num` jest równe zeru, albo nie udało się alokować pamięci; funkcja ustawia wtedy `errno` odpowiednio na `EINVAL` lub `ENOMEM`.

Wymagania funkcjonalne

Stan wejścia automatu ustala się za pomocą funkcji `ma_set_input` lub przez podłączenie tego wejścia do wyjścia automatu. Dopóki stan wejścia nie został ustalony jest nieustalony. Po odłączeniu wejścia automatu od wyjścia automatu stan tego wejścia jest nieustalony. Stan nieustalony sygnału oznacza, że nie znamy jego wartości.

Przy usuwaniu i łączeniu automatów należy zadbać o poprawne ich rozłączanie, aby nie pozostawały „wiszące” połączenia.

Biblioteka powinna zapewniać słabą gwarancję odporności na niepowodzenia alokacji pamięci, czyli nie powinna gubić pamięci, a wszystkie struktury danych powinny pozostawać w poprawnym stanie.

Wymagania formalne

Jako rozwiązanie zadania należy wstawić w Moodle archiwum zawierające plik `ma.c` oraz opcjonalnie inne pliki `*.h` i `*.c` z implementacją biblioteki, oraz plik `makefile` lub `Makefile`. Archiwum nie powinno zawierać innych plików ani podkatalogów, w szczególności nie powinno zawierać plików binarnych. Archiwum powinno być skompresowane programem `zip`, `7z` lub `rar`, lub parą programów `tar` i `gzip`. Po rozpakowaniu archiwum wszystkie pliki powinny się znaleźć we wspólnym podkatalogu.

Dostarczony w rozwiązaniu plik `makefile` lub `Makefile` powinien zawierać cel `libma.so`, tak aby polecenie `make libma.so` uruchamiało kompilowanie biblioteki i aby w bieżącym katalogu powstał plik `libma.so`. Polecenie to powinno również kompilować i dołączać do biblioteki załączony do treści zadania plik `memory_tests.c`. Należy opisać zależności między plikami i zapewnić, że kompilowane są tylko pliki, które zostały zmienione lub pliki, które od nich zależą. Wywołanie `make clean` powinno usuwać wszystkie pliki utworzone przez polecenie `make`. Plik `makefile` lub `Makefile` powinien zawierać pseudocel `.PHONY`. Może zawierać też inne cele, na przykład cel kompilujący i linkujący z biblioteką przykład jej użycia zawarty w załączonym do treści zadania pliku `ma_example.c` bądź cel uruchamiający testy.

Do kompilowania należy użyć `gcc`. Biblioteka powinna się kompilować w laboratorium komputerowym pod Linuksem. Pliki z implementacją biblioteki należy kompilować z opcjami:

```
-Wall -Wextra -Wno-implicit-fallthrough -std=gnu17 -fPIC -O2
```

Pliki z implementacją biblioteki należy linkować z opcjami:

```
-shared -Wl,--wrap=malloc -Wl,--wrap=calloc -Wl,--wrap=realloc  
-Wl,--wrap=reallocarray -Wl,--wrap=free -Wl,--wrap=strdup -Wl,--wrap=strndup
```

Opcje `-Wl,--wrap=` sprawiają, że wywołania funkcji `malloc`, `calloc` itd. są przechwytywane odpowiednio przez funkcje `__wrap_malloc`, `__wrap_calloc` itd. Funkcje przechwytywane są zaimplementowane w załączonym do treści zadania pliku `memory_tests.c`.

Implementacja biblioteki nie może gubić pamięci ani pozostawiać struktury danych w niespójnym stanie, również wtedy gdy wystąpił błąd alokowania pamięci. Poprawność implementacji będzie sprawdzana za pomocą programu [valgrind](#).

Implementacja nie może zawierać sztucznych ograniczeń na rozmiar przechowywanych danych – jedynymi ograniczeniami są rozmiar pamięci dostępnej w komputerze i rozmiar słowa maszynowego użytego komputera.

Ocena

Za w pełni poprawne rozwiązanie zadania implementujące wszystkie wymagania można zdobyć 20 punktów, z tego 14 punktów zostanie wystawionych na podstawie testów automatycznych, a 6 punktów to ocena jakości kodu. Za problemy ze skompilowaniem rozwiązania lub niespełnienie wymogów formalnych można stracić wszystkie punkty. Za ostrzeżenia wypisywane przez kompilator może być odjęte do 2 punktów.

Załączniki

Załącznikami do treści zadania są następujące pliki:

- `ma_example.c` – przykładowe testy biblioteki;
- `ma.h` – deklaracja interfejsu biblioteki;
- `memory_tests.c` – implementacja modułu biblioteki służącego do testowania reakcji implementacji na niepowodzenie alokowania pamięci;
- `memory_tests.h` – deklaracja interfejsu modułu biblioteki służącego do testowania reakcji implementacji na niepowodzenie alokowania pamięci.

Nie wolno modyfikować pliku `ma.h`. Zastrzegamy sobie możliwość zmiany testów oraz zawartości plików `memory_tests.h` i `memory_tests.c` podczas testowania rozwiązań.

PLIKI ZAŁĄCZONE DO TREŚCI ZADANIA

<code>ma_example.c</code>	9 marca 2025, 02:47
<code>ma.h</code>	20 lutego 2025, 15:44
<code>memory_tests.c</code>	4 lutego 2025, 18:49
<code>memory_tests.h</code>	4 lutego 2025, 18:49